



NanoPulldown: a Local *In Silico* Pulldown Screen

AlphaPulldown-style bait/prey screening with ESMFold2-Fast on a single Apple
Silicon Mac Studio

Etienne Reboul

Groupe Gibbérellines et adaptation à l'environnement
Institut de Biologie Moléculaire des Plantes (IBMP), Strasbourg

October 7, 2026

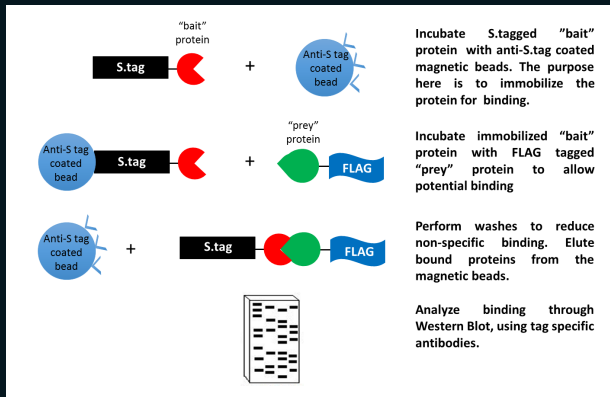
Outline

1. Introduction
 - Background
 - Problem statement
 - Language-model folding
 - Project Overview
2. The pipeline
3. Config and reuse
4. Report
5. Caveats and next steps

Introduction

Pull-down assay: finding what binds a bait

Background



Pull-down with tagged proteins (S-tag / FLAG). RaihaT, Wikimedia Commons, CC

BY-SA 3.0

1. **Pull-down:** tagged *bait* immobilized on beads
2. Incubate with *prey* proteins (purified, or a lysate)
3. Wash, then elute
4. Readout: Western blot, or **mass spectrometry (IP-MS)** for a proteome-wide list of candidate preys

Limits: direct and indirect binders are mixed.

The AlphaFold2 breakthrough

Background

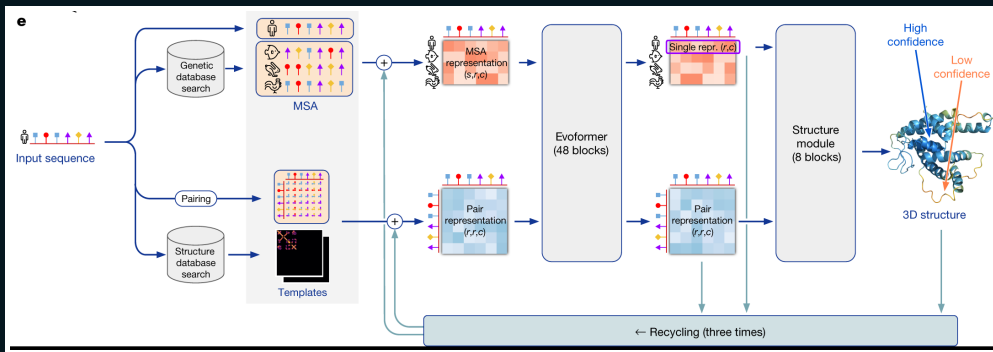


Fig. 1e, cropped; Wikimedia Commons, CC BY 4.0 (Jumper et al., 2021)

⇒ AF2 multimer is a **protein-protein docking engine** with confidence scores

John Jumper, Richard Evans, Alexander Pritzel, et al. *Highly accurate protein structure prediction with AlphaFold.*

Nature, 596:583–589, 2021.

URL <https://doi.org/10.1038/s41586-021-03819-2>

AlphaPulldown: the workflow

Background

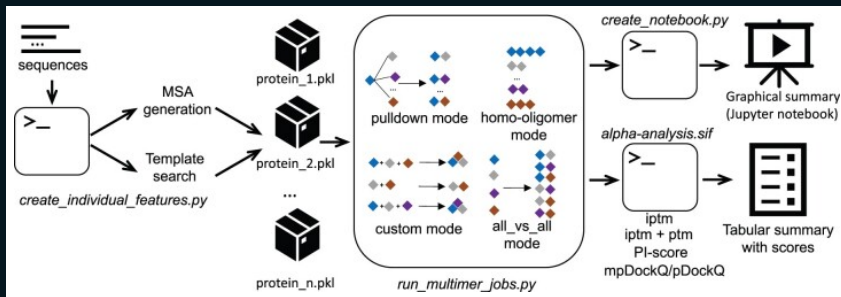


Fig. 1 of the paper; CC BY 4.0 (Yu et al., 2023)

⇒ AlphaPulldown automates identification of direct binding partners through the **in silico pulldown** with AF2-Multimer

Dingquan Yu, Grzegorz Chojnowski, Maria Rosenthal, and Jan Kosinski. [AlphaPulldown – a python package for protein–protein interaction screens using AlphaFold-Multimer](https://doi.org/10.1093/bioinformatics/btac749).

Bioinformatics, 39(1):btac749, 2023.

URL <https://doi.org/10.1093/bioinformatics/btac749>

Problem statement

Problem statement

- AlphaPulldown is **great**, but it assumes:
 - access to a GPU cluster :
 - ▶ Unmanageable to give access to a large number of users
 - ▶ Pangloss (ibmp cluster) does not have high-end GPUs for a full screen
 - ▶ High-end GPUs are expensive (around 12k EUR) currently not available through public procurement
 - MSA building :
 - ▶ Typically a slow process with Jackhammer and HHblits
 - ▶ Requires large sequence databases (UniRef90, MGnify, BFD, PDB70, etc.)
 - ▶ Not feasible on a laptop
 - AlphaFold models as docking engine :
 - ▶ AF2 multimer is no longer State-of-the-Art (SOTA) for protein-protein docking
 - ▶ AF3 weights are available on demand but have specific licensing restrictions

⇒ Can we run the same screen on a single MacStudio (range price : 3k-9k EUR)?

Protein LM as alternative folding engines

Language-model folding

Protein LMs (e.g. ESMFold, Lin et al., 2023): **LLM-style transformers** trained on proteins.

	AF2-Multimer	ESMFold-type (protein LM)
Input	sequence + MSA + templates	single sequence
Evolutionary signal	read from the alignment at inference	learned in the weights during pre-training
Pre-processing	database searches (slow, heavy)	none
Weak spot	few homologs $\Rightarrow$ poor MSA	orphan / novel sequences

Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, et al. [Evolutionary-scale prediction of atomic-level protein structure with a language model](#).

Science, 379(6637):1123–1130, 2023.

URL <https://doi.org/10.1126/science.ade2574>

ESMFold2-Fast designed for high-throughput screening Language-model

folding

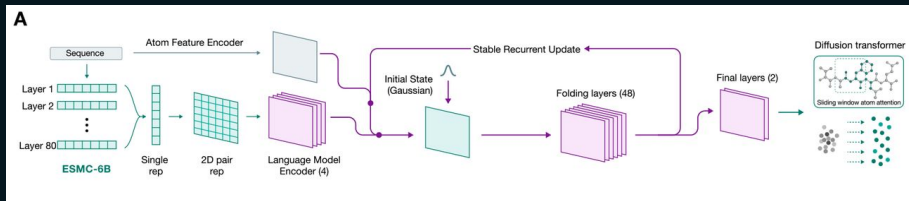


Fig. 2A, cropped, CC BY 4.0 (Candido et al., 2026)

- **ESMFold2:** recurrent folding trunk on **frozen ESMC** representations (pooled over all layers), diffusion transformer for atomic coordinates
- **Fast variant:** 24 folding layers instead of 48, built for high-throughput screening and design
- **No MSA:** sequence-only prediction competitive with MSA-based methods (slightly worse than AF3)
- ~9.4 s for 1024 residues on one H100 (reported 1.3× faster than AF3)

⇒ This is what makes a laptop pulldown screen feasible

Candido, Hayes, Derry, Rao, Lin, Verkuil, et al. *Language modeling materializes a world model of protein biology*. *bioRxiv*, 2026.

URL <https://doi.org/10.64898/2026.06.03.729735>

Motivation

Project Overview

- A pulldown / IP-MS experiment returns a **long list** of candidate interactors – which of them bind the bait *directly*?
 - Worked case: **BPM6** (AT3G43700, CUL3 substrate adaptor), GFP-IP in *A. thaliana* (PXD013906, Chico *et al.* 2020), 1 bait $\times$ $\sim$ 1 500 prey candidates
 - AlphaPulldown's UniProt-ID-driven bait/prey strategy answers this, but assumes AlphaFold-multimer/AF3, MSAs and a GPU cluster
- ⇒ Can we run the same screen **on one laptop**?

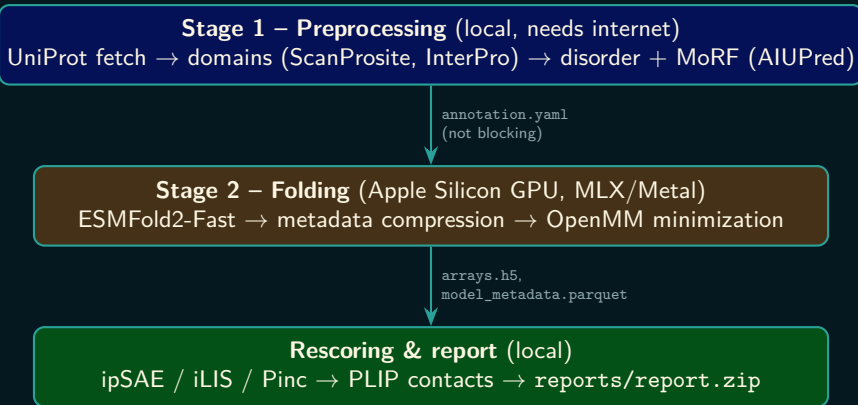
Design choices

Project Overview

1. **ESMFold2-Fast** (MLX/Metal): no MSA, no templates, no cluster
2. **Protein–protein only**: one seed, 1 diffusion sample per pair, no pose clustering
3. **One command, one report**: a single Snakemake rule `all` from sequence annotation to the final report
4. Adding pairs = a bait list + a prey list; pair specs are **generated**, not hand-written

The pipeline

Two stages, one diagram



Everything runs on one machine, in one Snakemake call.

Stage 1 - Preprocessing

- **Pair expansion:** `configs/baits.txt` $\times$ `configs/preys.txt` (AlphaPulldown syntax: `ID`, `ID:N`, `ID:start-stop`) $\rightarrow$ `configs/<bait>__<prey>.yaml`; pairs over `esmfold2.max_tokens` are skipped and listed
 - **Domain annotation:** ScanProsite + InterPro API (fast path by accession; InterProScan v6 MATCH as fallback)
 - **Disorder + MoRF:** AIUPred (docker), one container run per 200 sequences
 - Annotation runs **once per unique sequence** (cached) with polite, ramped rate limits on the public APIs
- $\Rightarrow$ proposed domains: block, curated **whenever convenient** - it never blocks folding

Stage 2 : Folding

- **ESMFold2-Fast** (biohub/ESMFold2-Fast), MLX on Metal: 10 loops, 100 sampling steps, 1 seed, 1 diffusion sample
- Memory-bound, not compute-bound: ~ 25 GB model load in unified memory $\Rightarrow$ `--resources gpu=1` is **mandatory**
- **Batched folding**: 50 pairs per model load, so the model is loaded once per chunk, not once per pair
- **Token limit**: peak memory grows $\sim n^{2.2}$; measured on a 36 GB M4 Max, 1 000 tokens ≈ 29 GB, 145 s (`max_tokens: 1000`)
- Metadata compression: pair ipTM / PAE $\rightarrow$ `arrays.h5` + `model_metadata.parquet`

Minimization, rescoring and contacts

1. **OpenMM minimization** (`amber14` + GBn2 implicit solvent; OpenCL with CPU fallback; divergence detection and failure table)
2. **Rescoring**: ipSAE, iLIS, Pinc (PAE cutoff 12 Å, $C\beta$ contact cutoff 8 Å) next to ESMFold2's own pair ipTM
3. **PLIP** (`docker, linux/amd64` under Rosetta): bait $\leftrightarrow$ prey contacts per minimized sample, merged into `plip_summary.csv` per pair
4. PAE-based **domain $\times$ domain confidence heatmap** per sample

Running it

```
conda env create -f envs/controller.yaml
python scripts/expand_pairs.py      # baits x preys -> configs/<pair>.yaml
# add the new pair stems to config.yaml's pairs: list
snakemake --use-conda --cores 4 --resources gpu=1 docker_heavy=3
```

- `gpu=1`: caps `run_esmfold2` at one model load – nothing else guards against concurrent loads (OOM)
- `docker_heavy=N`: keeps minimization / PLIP containers from running during a model load, then lets N run in parallel

⇒ **Output:** `one reports/report.zip`

Config and reuse

Adding a new screen

Edit **only**: the bait/prey lists, then one line per pair in `config.yaml`'s pairs:. **No script edits.**

file	role
<code>config.yaml</code>	shared defaults, both stages
<code>config.local.yaml</code>	per-machine overrides (gitignored): InterPro e-mail, <code>max_tokens</code>
<code>configs/baits.txt</code> , <code>preys.txt</code>	one protein spec per line
<code>configs/<bait>__<prey>.yaml</code>	chain spec + curated domains (generated)
<code>envs/</code>	one conda env per rule group (<code>--use-conda</code>)

Reused from `ab_initio_pipeline`

- Vended **ipSAE** and **Pinc** (`tools/`), unchanged
- Same dark `custom.css` report theme and `datavzrd` tables
- Same annotation logic (domains, disorder), adapted: **AIUPred** replaces **IUPred3/ANCHOR2** **and** **MoRFchibi2** in one step
- **Replaced**: no HPC scheduler, no ABCfold backends, no MSA, no ChimeraX / antechamber (OpenMM instead), no pose clustering

Report

One report, zipped

- Snakemake report, **SVG/PDF figures only**, dark theme (report/custom.css)
- **Zip, not bare .html**: browsers block the data: URLs needed to embed interactive datavzrd tables → unzip, open report.html
- Contents:
 - results table across all pairs (ipSAE / iLIS / Pinc / pair ipTM)
 - PLIP contact heatmaps, domain-aware
 - PAE domain × domain confidence heatmaps
 - minimization energy plots + failure table
- Stage-1 figures (domain map, disorder / binding) are kept as plain SVGs under data/annotation/<pair>/

Caveats and next steps

Assumptions still to confirm

- **MLX path:** assumed to fill `result.pae / pair_chains_iptm` like the Torch/CUDA path to check against a live run
- **AIUPred CLI** (`-i/-t/-o`) and the **InterProScan v6 MATCH** endpoint: to confirm against current docs
- **OpenMM OpenCL** on Metal: CPU fallback exists, but initialization is untested on other Macs
- **PLIP on Apple Silicon:** amd64 emulation is slow and needs Rosetta in Docker Desktop
- **Memory:** 3 concurrent PLIP containers once caused a swap/OOM crash $\Rightarrow$ start with `docker_heavy=2`

Roadmap

- Complete the BPM6 screen over the PXD013906 prey list and compare ranking against the PNAS Dataset S1 known / significant interactors
- Benchmark ESMFold2-Fast ranking metrics (ipSAE, iLIS, pair ipTM) on known positives vs. random preys
- Tune `max_tokens` and `pairs_per_load` per machine (`notebooks/esmfold2_memory_model.ipynb`)
- Optional: re-fold top hits with AF3 / ABCfold for confirmation

Questions?